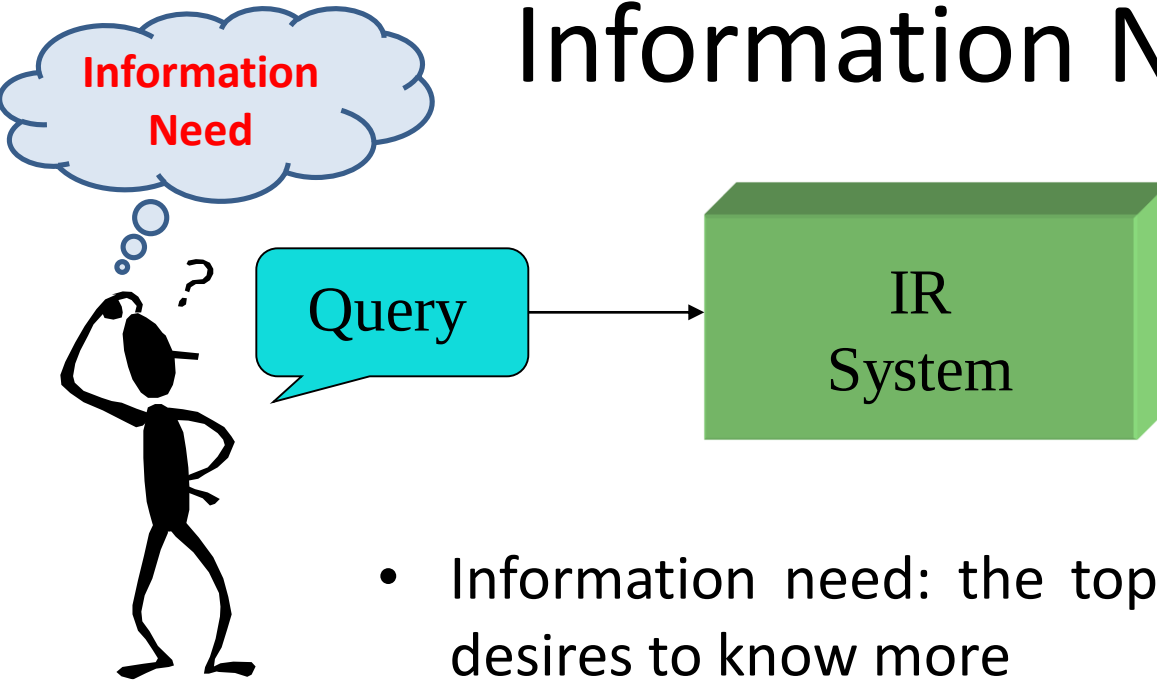


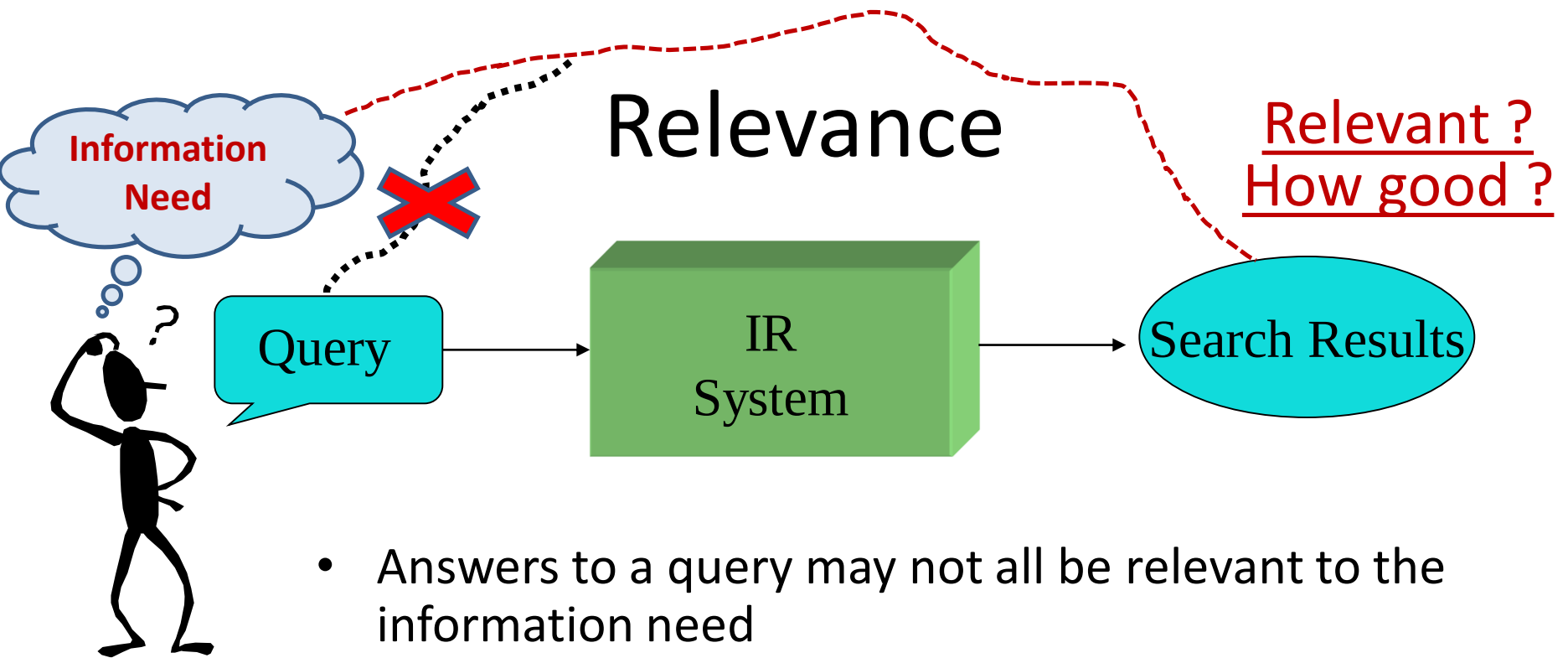
CS 419

Boolean Retrieval

Information Need vs Query

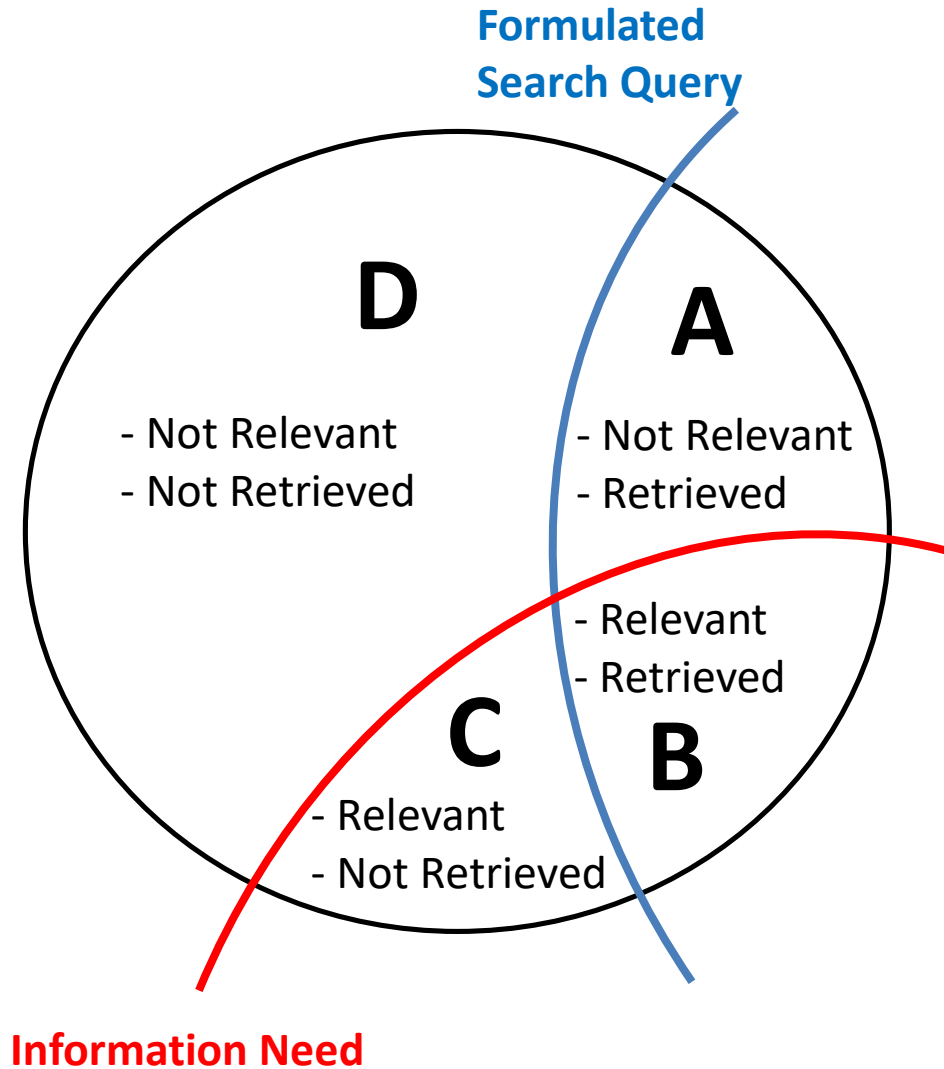


- Information need: the topic about which the user desires to know more
 - Unfortunately, often cannot be fed into a search engine
- Query: what the user conveys to the computer in an attempt to communicate the information need
 - Multiple queries may be formed to capture the same information need
 - A query may not capture the information need sufficiently

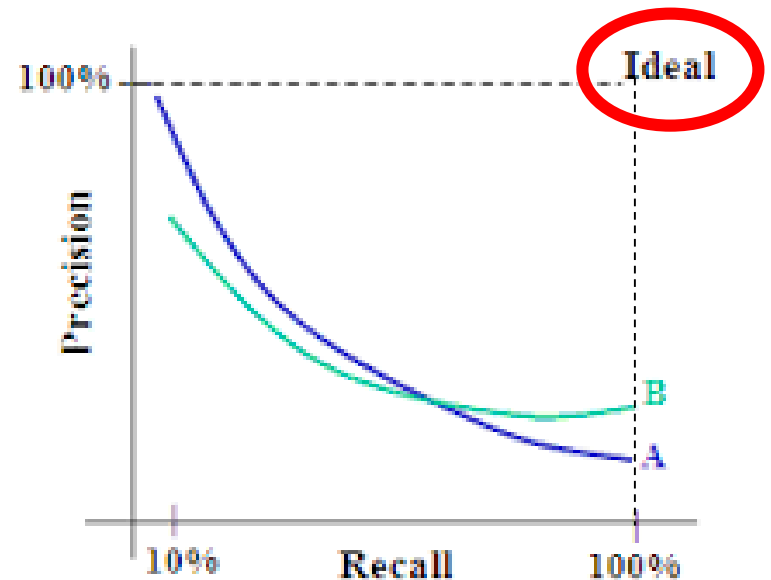


- Answers to a query may not all be relevant to the information need
- A document is relevant if it is one that the user perceives as containing information of value with respect to their information need
- How good are the returned answers?
 - **Precision:** the percentage of the returned results that are relevant to the information need
 - **Recall:** the percentage of the relevant documents in the collection that are returned

Precision and Recall



- Precision = ?
- Recall = ?



Precision and Recall

- Only return the exactly matched results? High precision.
- Return all documents? 100% recall
- Ideal: 100% precision, 100% recall
- In practice: we have to keep balance between precision and recall
- Critical point: for web search, which one is more important, precision or recall? Why?
 - Give an example in which 100% recall is required but accuracy can be traded off?
 - And, vice versa ?

Query Answering

- Which plays of Shakespeare contain the words “Brutus” and “Caesar” but not “Calpurnia”?
- Scan “Shakespeare’s Collected Works” once, less than 1 million words
- Is linear scan capable in all situations?
 - What if we have to search a large collection (e.g., the web) which contains billions or trillions of words?
 - How can we search for plays which contain “Brutus” and “Caesar” in the same sentence?
 - How can we rank the answers in relevance descending order?

Incidence Matrix

- Two dimensional: documents and terms
- Cell $M(t, d) = 1$ if term t appears in document d

		Documents					
		Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth ...
Terms	Antony	1	1	0	0	0	1
	Brutus	1	1	0	1	0	0
	Caesar	1	1	0	1	1	1
	Calpurnia	0	1	0	0	0	0
	Cleopatra	1	0	0	0	0	0
	mercy	1	0	1	1	1	1
	worser	1	0	1	1	1	0
	...						

Term and Document Vector

		Documents					Document vector	
		Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Terms	Antony	1	1	0	0	0	1	
	Brutus	1	1	0	1	0	0	
	Caesar	1	1	0	1	1	1	
	Calpurnia	0	1	0	0	0	0	
	Cleopatra	1	0	0	0	0	0	
	mercy	1	0	1	1	1	1	
	worser	1	0	1	1	1	0	
...								

Query Answering

- Query: Brutus AND Caesar AND NOT Calpurnia
 - $V_{\text{Calpurnia}} = 010000 \rightarrow \text{NOT } V_{\text{Calpurnia}} = 101111$
 - $V_{\text{Brutus}} \text{ AND } V_{\text{Caesar}} \text{ AND NOT } V_{\text{Calpurnia}}$
 $= 110100 \text{ AND } 110111 \text{ AND } 101111 = 100100$

	Documents					
	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth ...
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0
...						

Query Results

- Using the term vectors, we can only find whether the documents meet the query, but **cannot find which parts** of the documents meet the query

Antony and Cleopatra, Act III, Scene ii

Agrippa [Aside to Domitius Enobarbus]: Why, Enobarbus,
When Antony found Julius Caesar dead,
He cried almost to roaring; and he wept
When at Philippi he found Brutus slain.

Hamlet, Act III, Scene ii

Lord Polonius: I did enact Julius Caesar: I was killed i' the
Capitol; Brutus killed me.

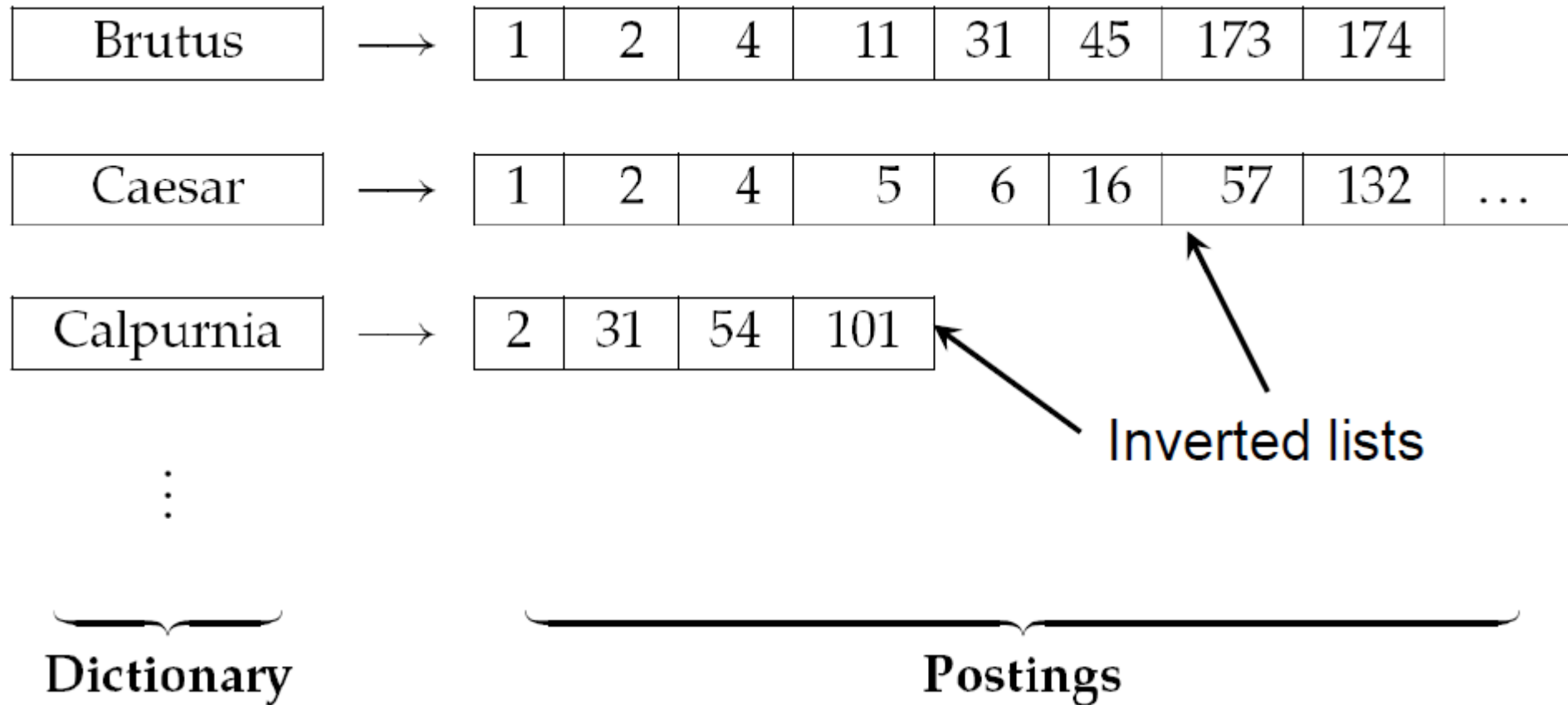
Boolean Retrieval Model

- We can pose any query which is in the form of a ***Boolean expression*** of terms, i.e., in which terms are combined with the operators AND, OR, and NOT
 - Each document is modeled as a set of words
- **Ad hoc retrieval**: retrieve documents that are relevant to an arbitrary user information need, communicated to the system by means of a one-off, user-initiated query

Compressing Incidence Matrices

- Suppose there are **1 million documents**, each of about **1,000 words**, and there are **500,000 distinct terms**
 - The incidence matrix has 500,000 rows and 1 million columns = 500 billion cells – too big to fit into main memory
- The matrix has no more than $1,000 \times 1 \text{ million} = 1 \text{ billion}$ 1's – **99.8% of the cells are zero (sparse)**
 - We can save a lot of space if we only store the 1 positions

Inverted Files



Doc 1

I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious:

Inverted Index

with *Doc Frequency*

term	docID	term	docID
I	1	ambitious	2
did	1	be	2
enact	1	brutus	1
julius	1	brutus	2
caesar	1	capitol	1
I	1	caesar	1
was	1	caesar	2
killed	1	caesar	2
i'	1	did	1
the	1	enact	1
capitol	1	hath	1
brutus	1	I	1
killed	1	I	1
me	1	i'	1
so	2	it	2
let	2	julius	1
it	2	killed	1
be	2	killed	1
with	2	let	2
caesar	2	me	1
the	2	noble	2
noble	2	so	2
brutus	2	the	1
hath	2	the	2
told	2	told	2
you	2	you	2
caesar	2	was	1
was	2	was	2
ambitious	2	with	2

term

doc. freq.

→

postings lists

ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
I	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

Building an Inverted Index

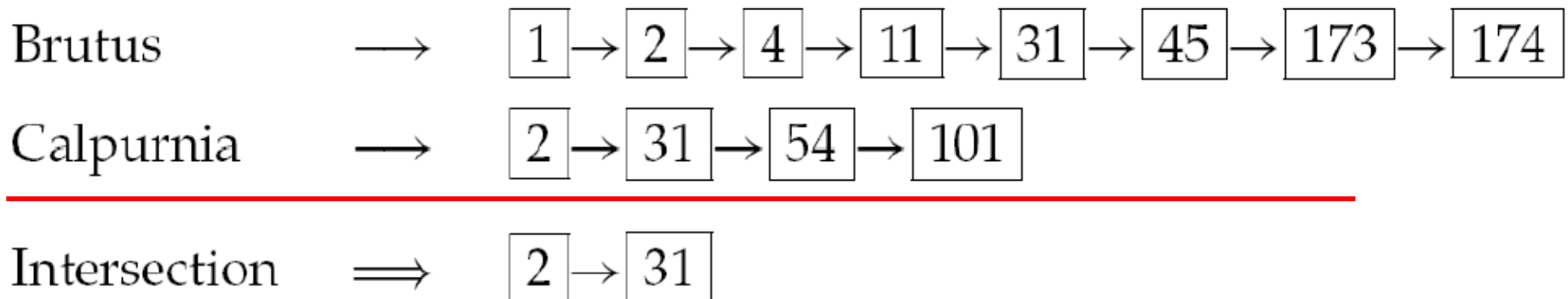
- Sorting according to document-ids
- Instances of the same term are grouped and split into a dictionary and postings
 - Can use either singly linked lists or variable length arrays
- The most efficient index for ad hoc search

Common **Pre**processing Steps

- Strip unwanted characters/markup
- Break into tokens (keywords) on whitespace.
- Stem tokens to “root” words
 - computational → comput
- Remove common **stopwords** (e.g. a, the, it, etc.).

Processing Boolean Queries

- Query: “Brutus AND Calpurnia”
- Steps
 - Locate Brutus in the dictionary, retrieve its postings
 - Locate Calpurnia in the dictionary, retrieve its posting
 - Intersect the two postings lists



Intersection of Two Posting Lists

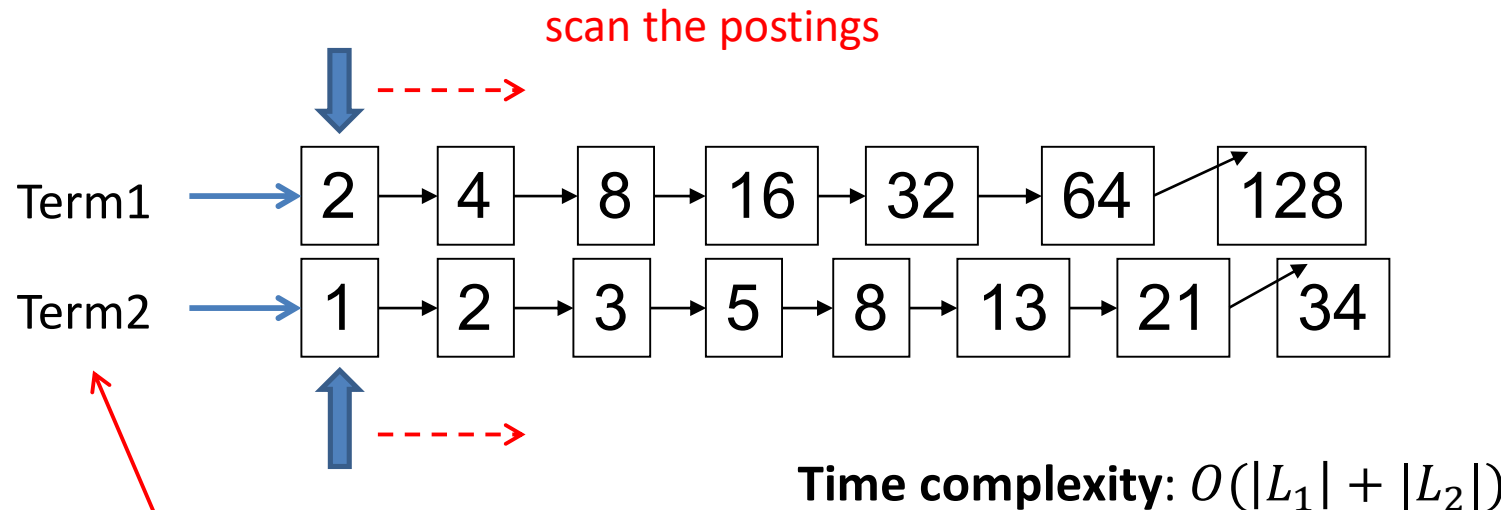
INTERSECT(p_1, p_2)

Similar to merge sort

```
1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(\text{answer}, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return answer
```

Intersection of Two Posting Lists

- Example: AND operation



Trick for speed-up: when performing multi-way join, starts from lowest frequency term to highest frequency ones

Conjunctive queries of >2 terms

INTERSECT($\langle t_1, \dots, t_n \rangle$)

- 1 $terms \leftarrow \text{SORTBYINCREASINGFREQUENCY}(\langle t_1, \dots, t_n \rangle)$
 - 2 $result \leftarrow \text{postings}(\text{first}(terms))$
 - 3 $terms \leftarrow \text{rest}(terms)$
 - 4 **while** $terms \neq \text{NIL}$ and $result \neq \text{NIL}$
 - 5 **do** $result \leftarrow \text{INTERSECT}(result, \text{postings}(\text{first}(terms)))$
 - 6 $terms \leftarrow \text{rest}(terms)$
 - 7 **return** $result$
-

Boolean Retrieval Model - **Pros**



- Popular retrieval model because:
 - Easy to understand for simple queries.
 - Clean formalism.
- Boolean models can be extended to include ranking.
- Reasonably efficient implementations possible for normal queries.

Boolean Retrieval Model - Cons



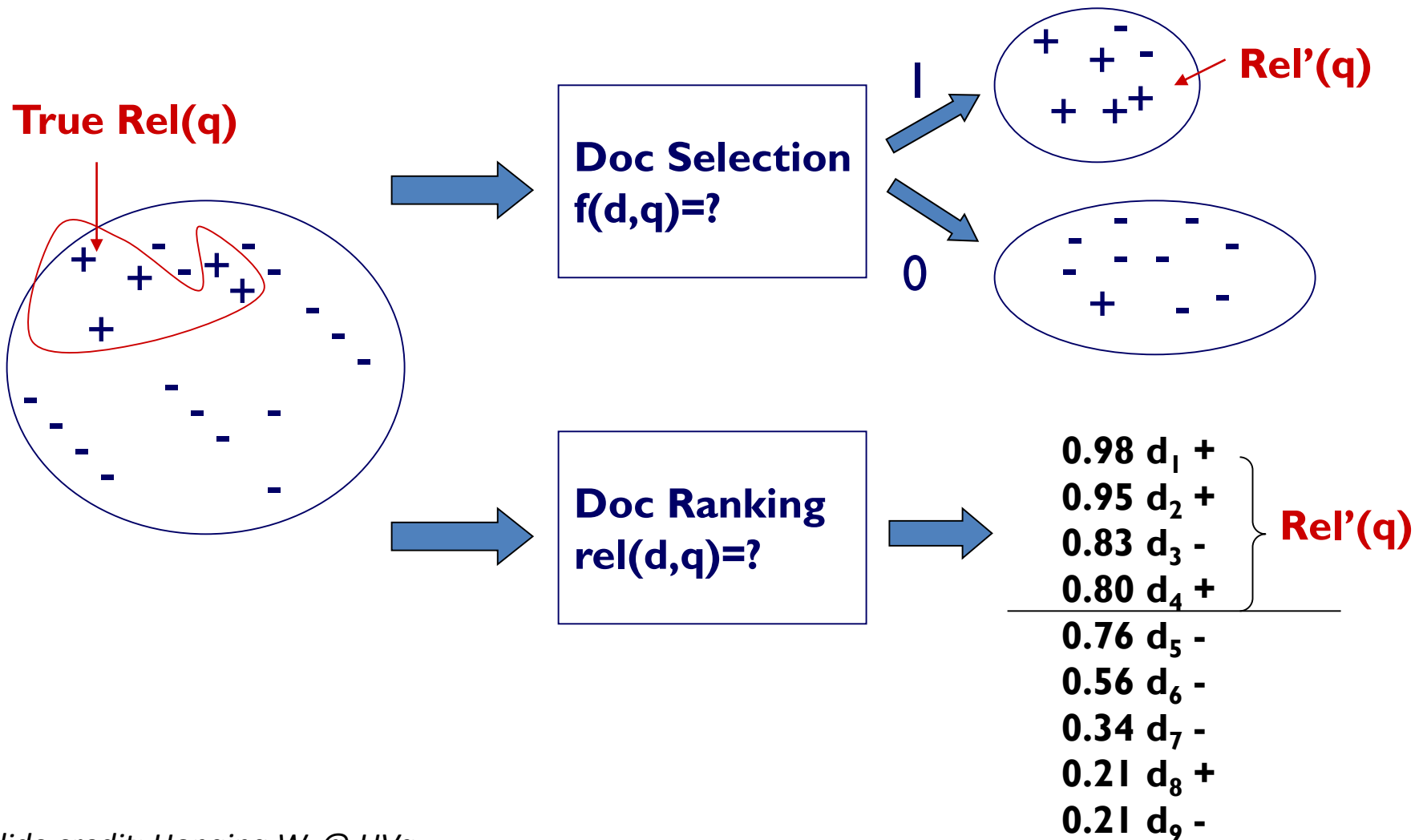
- The query is unlikely precise
 - “Over-constrained” query
(terms are too specific): no relevant documents found
 - “Under-constrained” query (terms are too general):
over delivery
 - It is hard to find the right position between these two extremes (hard for users to specify constraints)
- Even if it is accurate
 - Not all users would like to use such queries
 - All relevant documents are not equally relevant
 - No one would go through all the matched results

Boolean Retrieval Model - **Cons**



- Difficult to express **complex user requests**.
- Difficult to control the number of documents retrieved.
 - *All* matched documents will be returned.
- Difficult to **rank output**.
 - *All* matched documents logically satisfy the query.
- Difficult to perform relevance feedback.
 - If a document is identified by the user as relevant or irrelevant, how should the query be modified?

Document Selection vs. Ranking



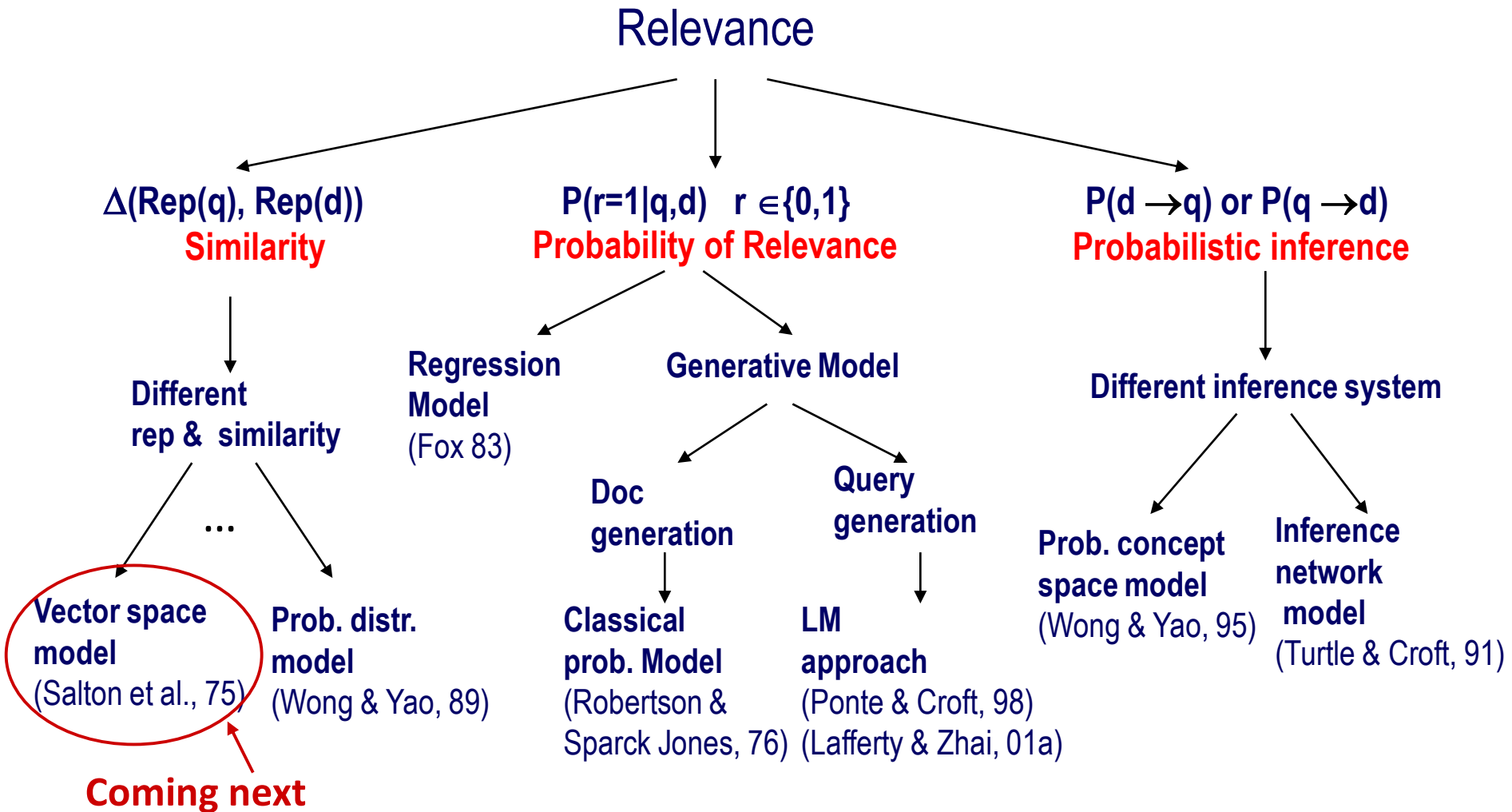
Ranking is often preferred

- Relevance is a matter of degree
 - Easier for users to find appropriate queries
- A user can stop browsing anywhere, so the boundary is controlled by the user
 - Users prefer coverage would view more items
 - Users prefer precision would view only a few

Retrieval procedure in modern IR

- Boolean model provides all the ranking candidates
 - Locate documents satisfying Boolean condition
 - E.g., “obama healthcare” -> “obama” OR “healthcare”
- Rank candidates by relevance
 - Important: the notation of relevance
- Efficiency consideration
 - Top-k retrieval ([Google](#))

Notion of relevance



CS 419

Vector Space Model

Some notations

- Vocabulary $V = \{w_1, w_2, \dots, w_N\}$ of language
- Query $q = t_1, \dots, t_m$, where $t_i \in V$
- Document $d_i = t_{i1}, \dots, t_{in}$, where $t_{ij} \in V$
- Collection $C = \{d_1, \dots, d_k\}$
- $\text{Rel}(q, d)$: relevance of doc d to query q
- $\text{Rep}(d)$: representation of document d
- $\text{Rep}(q)$: representation of query q

Relevance = Similarity

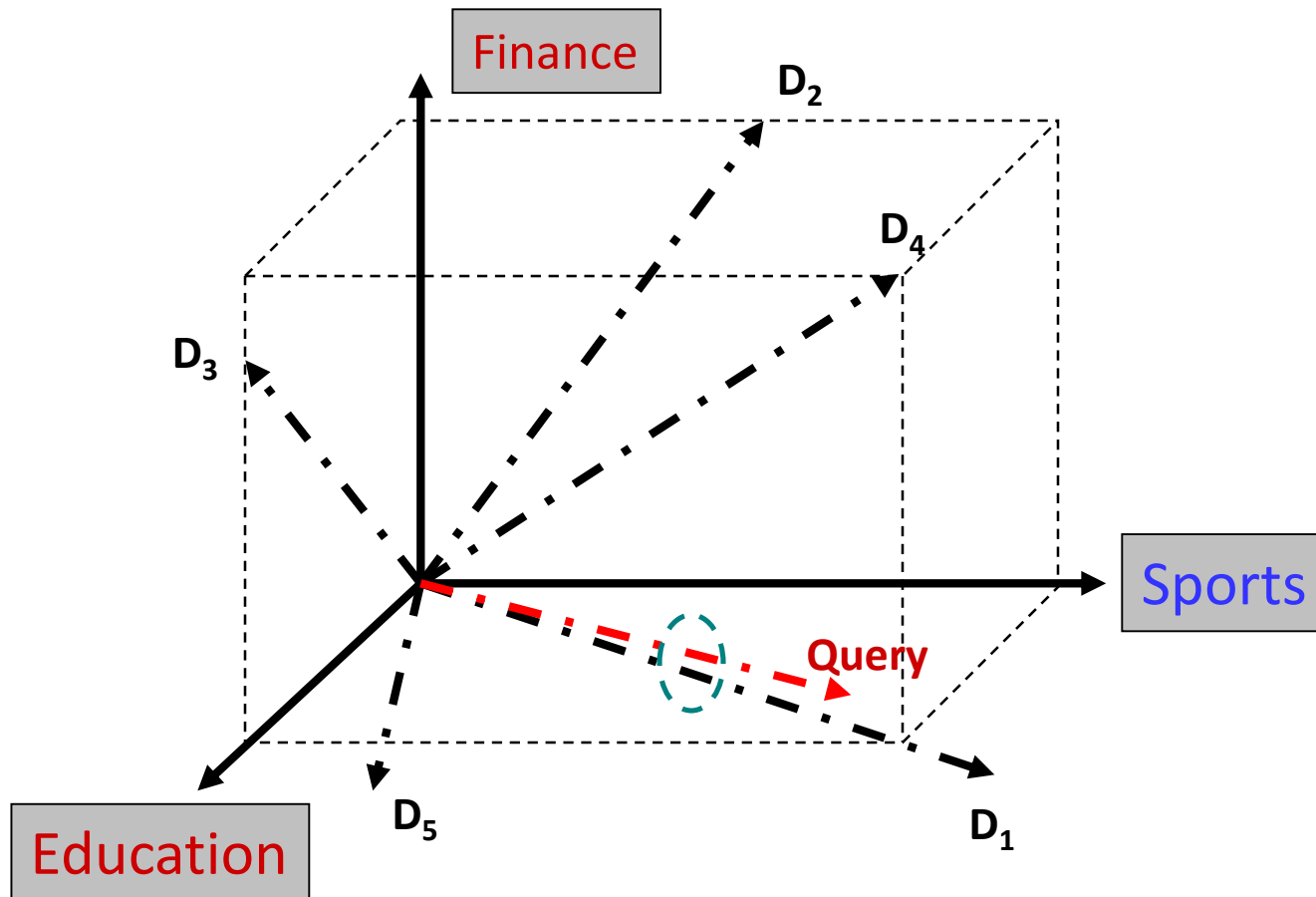
- Assumptions
 - Query and documents are represented in the same form
 - A query can be regarded as a “document”
 - $\text{Relevance}(d,q) \propto \text{similarity}(d,q)$
- $R(q) = \{d \in C \mid \text{rel}(d,q) > \theta\}$, $\text{rel}(q,d) = \Delta(\text{Rep}(q), \text{Rep}(d))$
- Key issues
 - How to represent query/document?
 - How to define the similarity measure $\Delta(x,y)$?

Vector space model

- Represent both doc and query by concept vectors
 - Each concept defines one dimension
 - K concepts define a high-dimensional space
 - Element of vector corresponds to concept weight
 - E.g., $d=(x_1, \dots, x_k)$, x_i is “importance” of concept i
- Measure relevance
 - Distance between the query vector and document vector in this concept space

VS Model: an illustration

- Which document is closer to the query?



What the VS model doesn't say

- How to define/select the “basic concept”
 - Concepts are assumed to be orthogonal
- How to assign weights
 - Weight in query indicates importance of the concept
 - Weight in doc indicates how well the concept characterizes the doc
- How to define the similarity/distance measure

What is a good “basic concept”?

- Orthogonal
 - Linearly independent basis vectors
 - “Non-overlapping” in meaning
 - No ambiguity
- Weights can be assigned automatically and accurately
- Existing solutions
 - Terms or N-grams, i.e., bag-of-words
 - Topics, i.e., topic model

How to assign weights?

- Important!
- Why?
 - Query side: not all terms are equally important
 - Doc side: some terms carry more information about the content
- How?
 - Two basic heuristics
 - TF (Term Frequency) = Within-doc-frequency
 - IDF (Inverse Document Frequency)

TF weighting

- Idea: a term is more important if it occurs more frequently in a document
- TF Formulas
 - Let $f(t, d)$ be the frequency count of term t in doc d
 - Raw TF: $tf(t, d) = f(t, d)$

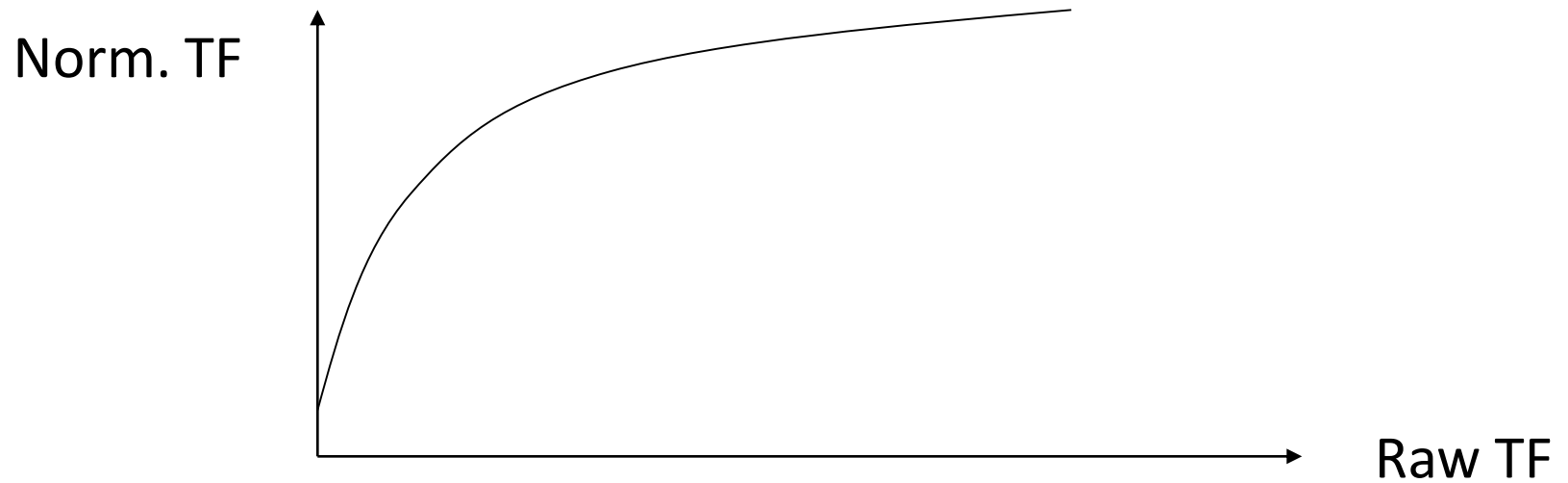
TF normalization

- Two views of document length
 - A doc is long because it is verbose
 - A doc is long because it has more content
- Raw TF is inaccurate
 - Document length variation
 - “Repeated occurrences” are less informative than the “first occurrence”
 - Relevance does not increase proportionally with number of term occurrence
- Generally penalize long doc, but avoid over-penalizing
 - Pivoted length normalization

TF normalization

- Sublinear TF scaling

$$- \quad tf(t, d) = \begin{cases} 1 + \log f(t, d), & \text{if } f(t, d) > 0 \\ 0, & \text{otherwise} \end{cases}$$

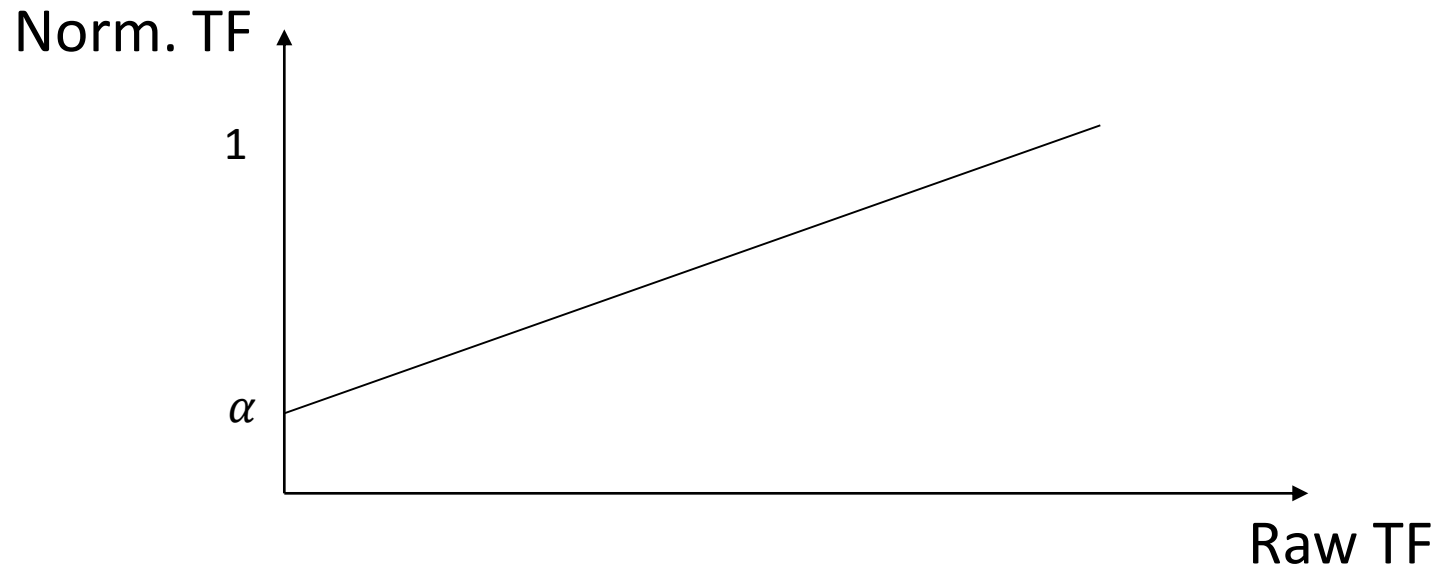


TF normalization

- Maximum TF scaling

- $tf(t, d) = \alpha + (1 - \alpha) \frac{f(t, d)}{\max_t f(t, d)}$

- Normalize by the most frequent word in this doc



Document frequency

- Idea: a term is more discriminative if it occurs only in fewer documents

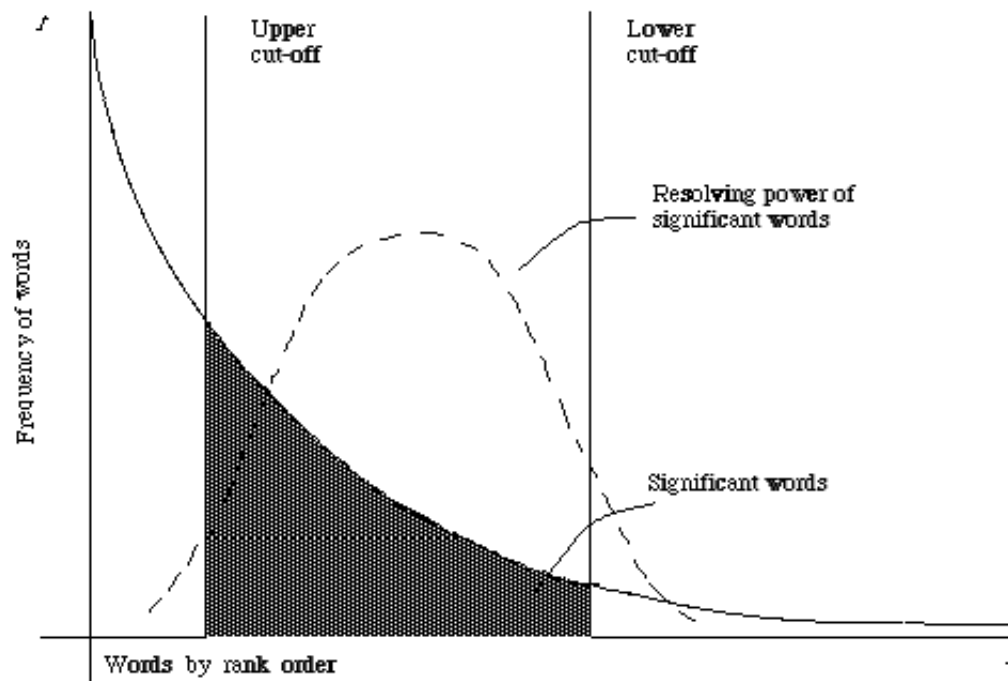


Figure 2.1. A plot of the hyperbolic curve relating f , the frequency of occurrence and r , the rank order (Adapted from Schultz⁴⁴ page 120)

IDF weighting

- Solution

- Assign higher weights to the rare terms

- Formula

- $IDF(t) = 1 + \log\left(\frac{N}{df(t)}\right)$

Non-linear scaling

Total number of docs in collection

Number of docs containing term t

- A corpus-specific property

- Independent of a single document

Why document frequency

- How about total term frequency?

- $ttf(t) = \sum_d f(t, d)$

Table 1. Example total term frequency v.s. document frequency in Reuters-RCV1 collection.

Word	ttf	df
try	10422	8760
insurance	10440	3997

- Cannot recognize words frequently occurring in a subset of documents

TF-IDF weighting

- Combining TF and IDF
 - Common in doc $\rightarrow$ high tf $\rightarrow$ high weight
 - Rare in collection $\rightarrow$ high idf $\rightarrow$ high weight
 - $w(t, d) = TF(t, d) \times IDF(t)$
- Most well-known document representation schema in IR! (G Salton et al. 1983)



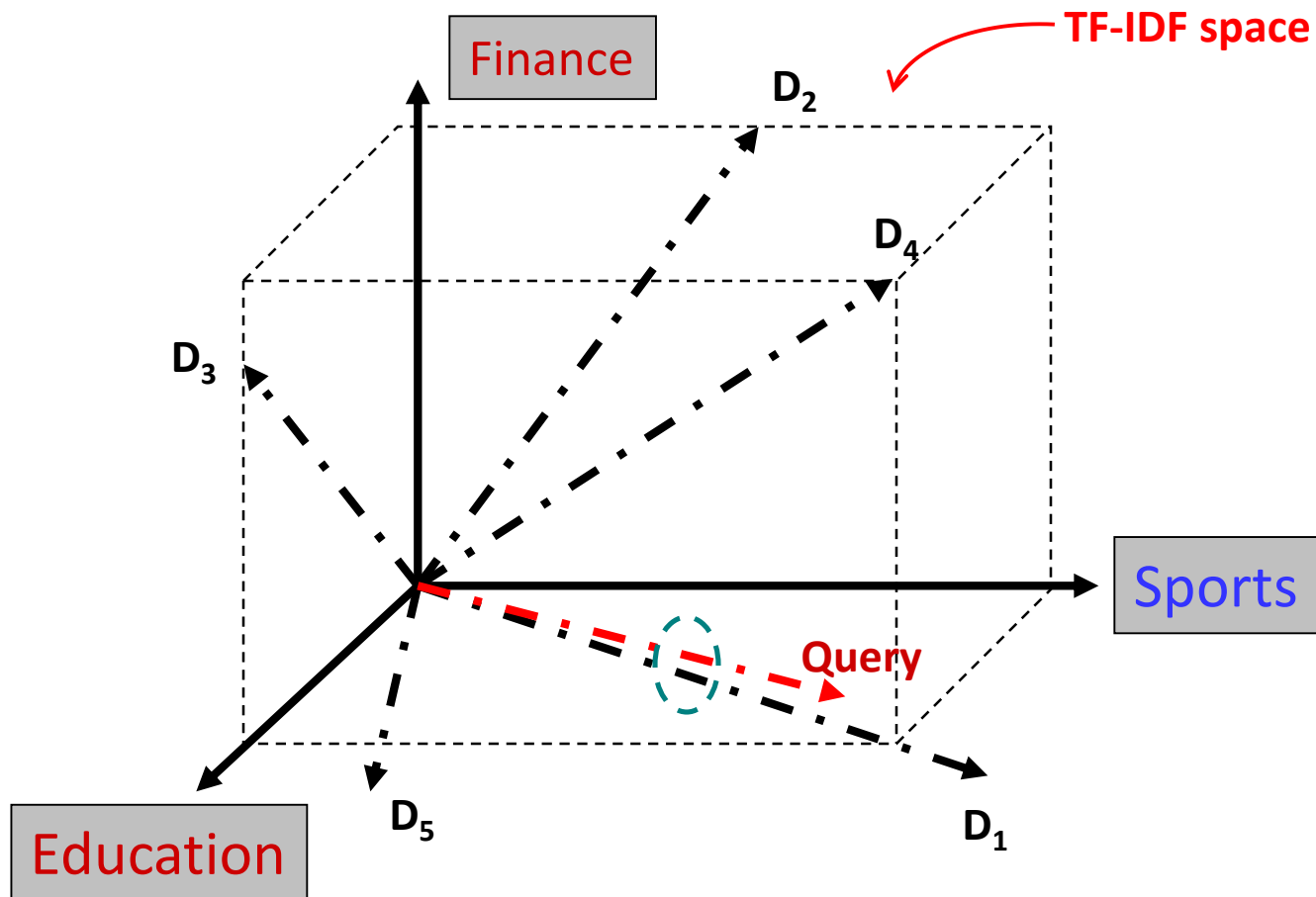
"Salton was perhaps the leading computer scientist working in the field of information retrieval during his time." - wikipedia

[Gerard Salton Award](#)

– highest achievement award in IR

How to define a good similarity measure?

- Euclidean distance?



How to define a good similarity measure?

- Euclidean distance

- $dist(q, d) =$

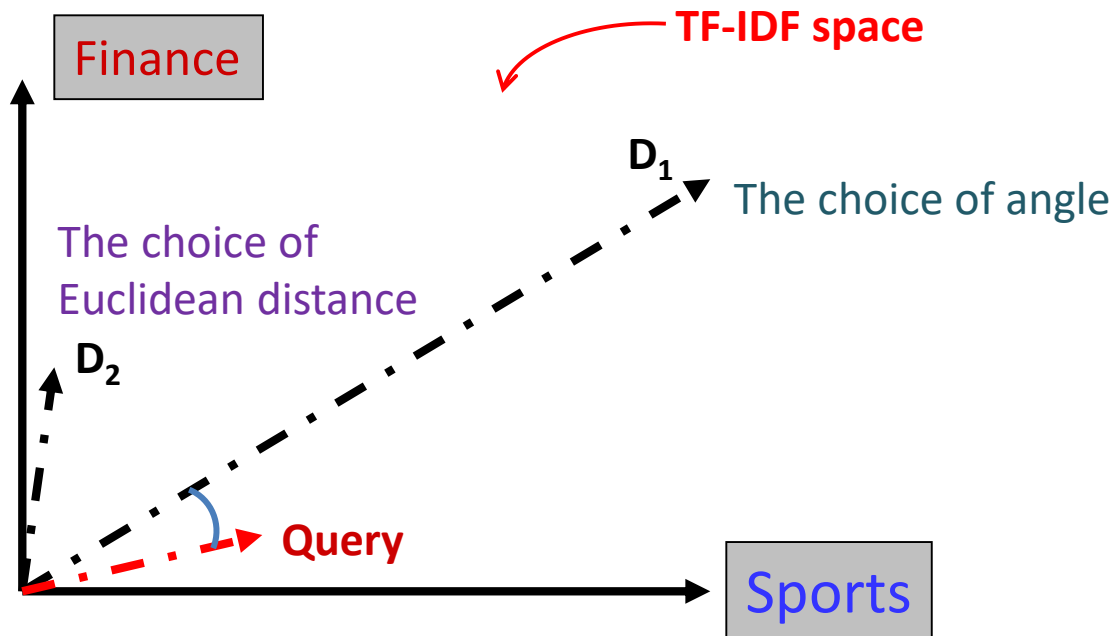
- $$\sqrt{\sum_{t \in V} [tf(t, q)idf(t) - tf(t, d)idf(t)]^2}$$

- Longer documents will be penalized by the extra words

- We care more about how these two vectors are overlapped

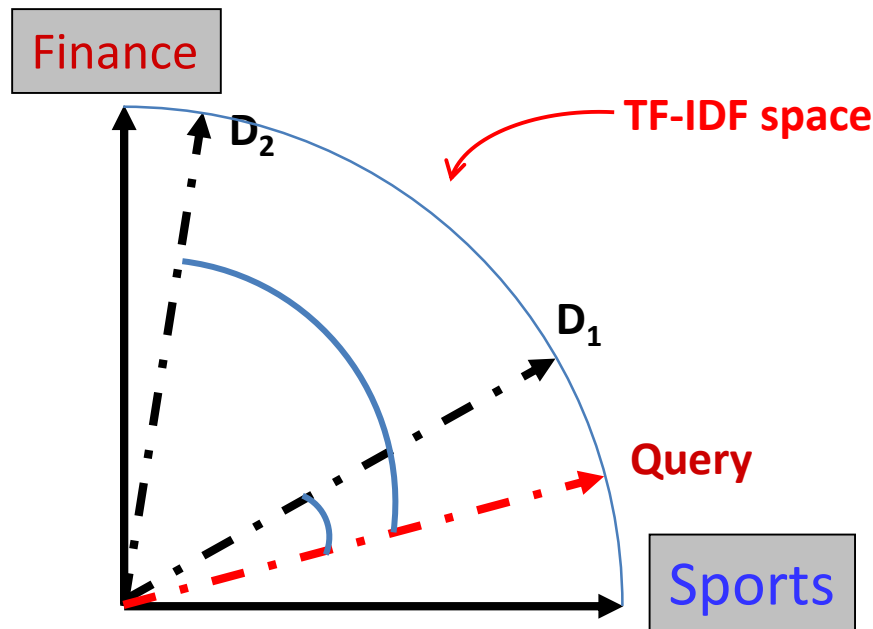
From distance to angle

- Angle: how vectors are overlapped
 - Cosine similarity – projection of one vector onto another



Cosine similarity

- Angle between two vectors
 - $\text{cosine}(V_q, V_d) = \frac{V_q \times V_d}{|V_q|_2 \times |V_d|_2} = \boxed{\frac{V_q}{|V_q|_2}} \times \frac{V_d}{|V_d|_2}$
 - Document length normalized
- TF-IDF vector
- Unit vector



Fast computation of cosine in retrieval

- $\text{cosine}(V_q, V_d) = V_q \times \frac{V_d}{|V_d|_2}$
 - $|V_q|_2$ would be the same for all candidate docs
 - Normalization of V_d can be done in index time
 - Only count $t \in q \cap d$
 - Score accumulator for each query term when intersecting postings from inverted index

Fast computation of cosine in retrieval

- Maintain a score accumulator for each doc when scanning the postings

Query = “info security”

$S(d,q) = g(t_1) + \dots + g(t_n)$ [sum of TF of matched terms]

Info: (d1, 3), (d2, 4), (d3, 1), (d4, 5)

Security: (d2, 3), (d4, 1), (d5, 3)

Can be easily applied to TF-IDF weighting!

Accumulators:		d1	d2	d3	d4	d5
info	(d1,3) =>	3	0	0	0	0
	(d2,4) =>	3	4	0	0	0
	(d3,1) =>	3	4	1	0	0
	(d4,5) =>	3	4	1	5	0
security	(d2,3) =>	3	7	1	5	0
	(d4,1) =>	3	7	1	6	0
	(d5,3) =>	3	7	1	6	3

Keep only the most promising accumulators for top K retrieval

Advantages of VS Model

- Intuitive
- Easy to implement
- Well-studied/Mostly evaluated
- The Smart system
 - Developed at Cornell: 1960-1999
 - Still widely used
- Warning: Many variants of TF-IDF!

Disadvantages of VS Model

- Assume term independence
- Assume query and document to be the same
- Lack of “predictive adequacy”
 - Arbitrary term weighting
 - Arbitrary similarity measure
- Lots of parameter tuning!

What you should know

- Document ranking v.s. selection
- Basic idea of vector space model
- Two important heuristics in VS model
 - TF
 - IDF
- Similarity measure for VS model